SMART CAMPUS ANALYTICS USING PYTHON HADOOP STREAMING

Guide to Completing the Remaining Tasks

Status as of 11 October 2026 | Repository: SmartCampusAnalytics (branch main)

Where the project stands

All six MapReduce jobs are implemented, tested and validated locally. The Python scripts are complete, and none of the remaining tasks requires code changes. What is left is evidence and submission work: run the jobs on real Hadoop, capture screenshots, assemble the submission and prepare for the viva.

Status	What it covers
A. Verified locally	All six jobs. 36 automated tests pass (0 failed, 0 skipped). 21 validation invariants pass. Every output row matches an independent reference implementation. Results are identical with and without the combiner and with 1-4 reducers.
B. Verified on Hadoop	**Nothing yet.** Hadoop was not installed on the development machine. Task 1 closes this gap.
C. Not yet verified	Real Hadoop behaviour: `-files` shipping, the Job 6 partitioner options, the Job 5 distributed-cache file, and whether Hadoop appends a TAB to output lines (review finding F6).

Remaining tasks

#	Task	Time	Outcome
1	Run all six jobs on real Hadoop (standalone mode)	30-45 min	Status B becomes verified
2	Capture execution and output screenshots	15 min	Worksheet section 13 evidence
3	Assemble the submission package	20 min	Everything the instructor asked for
4	Prepare for the viva	1-2 h	Confident answers to the viva questions
5	Final check and push	10 min	Clean, reproducible repository

Before you start. Open a terminal in the project folder. Every command in this guide assumes you are there:
`cd "/Users/deepesh/Developer/Big Data project"`

Task 1: Run the jobs on real Hadoop

Hadoop has a **standalone mode** that runs the real Hadoop Streaming engine on your laptop against local files. It needs no cluster and no HDFS. It tests exactly what the local simulator cannot: Hadoop's own key sorting, shipping of scripts with `-files`, the distributed-cache side file in Job 5, the Job 6 partitioner, and Hadoop's output format.

Step 1.1: Install Java and Hadoop

Hadoop 3 runs on Java 8 or 11. Install both with Homebrew:

```
brew install openjdk@11 hadoop
export JAVA_HOME="$(brew --prefix openjdk@11)"
hadoop version                  # must print "Hadoop 3.x.y"
```

To make `JAVA_HOME` permanent, add the export line to `~/.zshrc`. A warning such as *Unable to load native-hadoop library* is normal on macOS. Ignore it.

Step 1.2: Refresh the local results

The Hadoop results are compared with `outputs/`, so make sure those are current:

```
python3 scripts/generate_data.py      # seed 42, identical to the committed data
python3 scripts/run_pipeline.py       # expect 21 [PASS] lines, exit code 0
```

Step 1.3: Run all six jobs on Hadoop

One script runs all nine Hadoop jobs (Jobs 4 and 5 have several stages), with the combiner and the Job 6 partitioner, and then compares every result with the local output:

```
scripts/run_hadoop.sh                 # writes to /tmp/smartcampus_hadoop
```

Expected result (about 1-3 minutes):

```
== job1
...
== job6

== Comparison with local outputs/ (trailing TAB stripped)
job1 MATCH (113 lines incl. header)
job2 MATCH (13 lines incl. header)
job3 MATCH (13 lines incl. header)
job4 MATCH (67 lines incl. header)
job5 MATCH (7 lines incl. header)
job6 MATCH (785 lines incl. header)
Trailing TAB on reducer output lines: YES (review finding F6 confirmed)   <- or NO
```

Success criterion. All six lines say **MATCH** and the script exits with code 0 (`echo $?` prints 0). A summary is saved to `/tmp/smartcampus_hadoop/compare.txt`, and per-job Hadoop logs, including the SmartCampus rejection counters, are in `/tmp/smartcampus_hadoop/logs/`.

Honesty note. The script was tested only against a stand-in that imitates Hadoop Streaming. Your run is the first on real Hadoop, which is exactly why it is worth doing.

Step 1.4: If something fails

Symptom	Cause and fix
`hadoop not found on PATH`	Hadoop is not installed or the shell has not reloaded. Run `brew install hadoop` and open a new terminal.
`JAVA_HOME is not set`	Run `export JAVA_HOME="$(brew --prefix openjdk@11)"`.
`hadoop-streaming jar not found`	Locate it with `find $(brew --prefix hadoop) -name 'hadoop-streaming-*.jar'`, then run `HADOOP_STREAMING_JAR=<path> scripts/run_hadoop.sh`.
Job fails with `python3: not found` or exit code 127	The Hadoop task cannot see your Python. Run `PYTHON=$(which python3) scripts/run_hadoop.sh`.
`ModuleNotFoundError: mr_common`	The shared module was not shipped. The script ships it with every job, so report this: it would be a real compatibility finding.
Job fails with `PipeMapRed.waitOutputThreads(): subprocess failed`	A Python script crashed inside Hadoop. The Python traceback is in the job's log under `/tmp/smartcampus_hadoop/logs/`.
A job says **DIFFERS**	Open `/tmp/smartcampus_hadoop/logs/jobN.diff`. Lines starting with < came from Hadoop and lines starting with > from the local run. Save the diff and report it.

Step 1.5: Record the result

Once all six jobs match, update the verification table in `README.md`: move the Hadoop items from row C to row B. Name the Hadoop version (from `hadoop version`) and whether the trailing TAB appeared. Then commit:

```
git add README.md
git commit -m "docs: record Hadoop standalone verification (Hadoop 3.x.y)"
```

Optional, for higher marks: a pseudo-distributed (HDFS) run. Standalone mode proves Streaming compatibility. A full single-node HDFS setup additionally shows the `hdfs dfs -put / -cat` workflow. Follow the official Apache guide *Hadoop: Setting up a Single Node Cluster* (pseudo-distributed operation), then use the manual commands in the README's Hadoop section with `/campus/in` paths.

Task 2: Capture the screenshots

Worksheet section 13 asks for *screenshots of execution and output*. Take each one with **Cmd + Shift + 4**, then **Space**, then click the terminal window. Make the terminal wide enough that lines do not wrap, and save the images to a `screenshots/` folder with the suggested names.

#	Command to run	What must be visible	File name
1	`python3 scripts/generate_data.py`	Four datasets with row counts	01_generate.png
2	`head -5 data/*.csv`	Headers and sample rows of all four datasets	02_datasets.png
3	`python3 src/job1_wifi_mapper.py < data/wifi_logs.csv 2>/dev/null | head`	Mapper key/value pairs (worksheet section 7, step 4)	03_mapper.png
4	`python3 src/job1_wifi_mapper.py < data/wifi_logs.csv 2>/dev/null | LC_ALL=C sort | python3 src/job1_wifi_reducer.py | head`	Reducer output after sort (section 7, step 6)	04_reducer.png
5	`python3 scripts/run_pipeline.py`	All 21 [PASS] lines	05_pipeline.png
6	`python3 -m unittest discover -s tests -v`	Final lines: Ran 36 tests ... OK	06_tests.png
7	`cat outputs/job1_peak_hours.csv outputs/job2_access_points.csv`	Jobs 1 and 2 results	07_job1_2.png
8	`cat outputs/job3_canteen_items.csv outputs/job4_top_pairs.csv`	Jobs 3 and 4 results	08_job3_4.png
9	`cat outputs/job5_overall.csv outputs/job5_anomalies.csv`	Job 5 results	09_job5.png
10	`head -15 outputs/job6_route_hour_speed.csv; grep -c ',CONGESTED' outputs/job6_route_hour_speed.csv`	Job 6 results and congested count	10_job6.png
11	`scripts/run_hadoop.sh`	Hadoop run with six MATCH lines	11_hadoop.png
12	`cat /tmp/smartcampus_hadoop/job1/part-00000 | head`	A real Hadoop part file	12_hadoop_part.png

Never stage or edit a screenshot. If a command's output differs from this guide, capture what really happened and note the difference. Genuine evidence is worth more than a perfect picture.

Task 3: Assemble the submission package

Map each item the worksheet requires (section 13) to what already exists in the repository:

Requirement	Where it is	Action needed
Source code for all jobs	`src/` (18 job scripts, combiner, shared module)	None
Input datasets used	`data/*.csv` (synthetic, seed 42)	None
Screenshots	`screenshots/`	Task 2
Completed worksheet (Q1-Q15)	`docs/WORKSHEET.md`	Copy the answers onto the official worksheet in your own words
Expected output worksheet (section 9)	`docs/WORKSHEET.md`, table in section 9	Copy into the official table
Result interpretation for every job	`docs/WORKSHEET.md`, 'Result interpretation'	Read and rephrase
Performance paragraph	`docs/WORKSHEET.md`, 'Performance considerations'	Read and rephrase
Viva participation	Task 4	Prepare

Recommended package layout

```
SmartCampusAnalytics_<YourName>_<RollNo>.zip
  src/  scripts/  tests/  data/  outputs/  docs/  screenshots/
  README.md  PRD.md  requirements.txt
  Worksheet_<YourName>.pdf         <- the official worksheet, filled in
```

Create the zip from a clean state so it contains no caches or OS files:

```
git add screenshots && git commit -m "docs: add execution screenshots"
git status                       # must print nothing
git archive --format=zip -o ../SmartCampusAnalytics_submission.zip HEAD
# git archive packs committed files only, so commit everything first
```

Academic integrity. The instructor note says students should implement Jobs 2-6 themselves. Make sure you understand every script and can explain it line by line, and follow your course's policy on declaring AI assistance. The commit history credits the assistant openly. Keep it that way.

Task 4: Prepare for the viva

Read `docs/WORKSHEET.md` (answers), `docs/ARCHITECTURE.md` (design) and `docs/REVIEW.md` (defects found). Then make sure you can explain the points below without notes. They are the parts of this project that examiners are most likely to probe, and the ones that show real understanding.

Topic	What to be able to say
The reference reducer was wrong	Hadoop delivers one `key<TAB>value` line per record, sorted by key. It never joins values with ';'. The worksheet's reducer also reset its totals on every line. On real data it printed one line for the whole dataset instead of 112 hourly rows (REVIEW F1).
How reducers find key boundaries	`itertools.groupby` over sorted lines: when the key changes, the previous group is complete, so memory stays O(1) per key.
Why every value carries a count	Averages are computed from (sum, count). A combiner can then pre-add partial results safely, any number of times. An average of averages would be wrong.
Job 4: two stages	Stage 1 groups by TransactionID, takes the distinct items, sorts them and emits each pair once. Stage 2 counts pairs across transactions. Sorting makes (Coffee, Sandwich) and (Sandwich, Coffee) the same key.
Job 5: passing the global average	Jobs share no memory. The global stage writes one line to a file, and the filter stage reads it (locally via GLOBAL_STATS, on Hadoop via the distributed cache). The comparison is strictly greater-than and exact: 2 x total x G_visits > 3 x G_total x visits.
Job 6: why `RouteID|Hour`	Hadoop's default key is the text before the first TAB. With `RouteID<TAB>Hour`, Hadoop would group by route only and break the averages. A single field joined with `|` is correct by default. This was found in the review and proven with a test (REVIEW F2).
Malformed records	Each row is validated, rejected rows are counted by reason as Hadoop counters, and records = valid + rejected is checked.
How correctness was proven	An independent reference implementation, 36 tests, mutation testing (7 deliberate bugs, all caught), and identical results across combiner and reducer settings.

Practice questions (worksheet section 10)

Answer each one aloud in under a minute. Short answers are at the end of `docs/WORKSHEET.md`.

☐	What are the three major phases of MapReduce? Why does Hadoop sort mapper output?
☐	What is a combiner, and when is it safe? Which of our jobs use one?
☐	What is a partitioner? Describe the custom partitioner for Job 6.
☐	Can a job have zero reducers? What happens with multiple reducers?
☐	What is a composite key? Give the Job 6 example.
☐	What is the effect of an excessively large key group? How does data skew show up here?
☐	How would you test a mapper independently? (Show: cat file | python3 mapper.py)
☐	How would the solution change for millions of records? (Cite the 1M-row benchmark: the combiner cut one map task's shuffle from 249,981 to 119 records.)

Live demo you should be able to perform

```
cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py 2>/dev/null | head -3
cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py 2>/dev/null \
    | LC_ALL=C sort | python3 src/job1_wifi_reducer.py | head -3
python3 scripts/run_pipeline.py --combiner --reducers 3   # same results, different execution
```

Task 5: Final check before submission

Run these three commands from a clean state. All three must succeed:

```
python3 -m compileall -q src scripts tests && echo compile-ok
python3 scripts/run_pipeline.py --run-tests ; echo "exit=$?"     # exit=0
git status --short                                               # empty
```

Submission checklist

☐	Hadoop run: six MATCH lines, README verification table updated (Task 1)
☐	Twelve screenshots saved in screenshots/ (Task 2)
☐	Official worksheet filled in by hand or typed, Q1-Q15 and the section 9 table (Task 3)
☐	One result interpretation per job and the performance paragraph included (Task 3)
☐	You can explain every script and the eight viva topics without notes (Task 4)
☐	Pipeline exit 0, 36 tests OK, compile-ok (Task 5)
☐	All changes committed. Pushed to GitHub if your course uses it: git push origin main
☐	Submission zip created and opened once to confirm it contains everything

Command reference

Purpose	Command
Regenerate datasets	`python3 scripts/generate_data.py [--seed N]`
Run all jobs locally	`python3 scripts/run_pipeline.py`
Extension-challenge mode	`python3 scripts/run_pipeline.py --combiner --map-tasks 4 --reducers 3`
Run the tests	`python3 -m unittest discover -s tests -v`
Run on Hadoop and compare	`scripts/run_hadoop.sh`
Validation report	`outputs/validation_report.json`

This guide reflects the repository at the commit that added it. If a command's output differs, trust what the command actually prints, and record the difference rather than hiding it.